Azure DevOps Engineer – Take-Home Assignment

Purpose

This assignment is designed to help us understand how you approach CI/CD design for a commercial Windows desktop application.

We do **not** expect a perfect, production-ready, fully working solution. Instead, we are interested in your concept, structure, priorities, and engineering decisions you consider important. Please focus on showing your reasoning and practical understanding.

Scenario

Design a CI/CD process for a commercial Windows desktop application developed in .NET.

The solution consists of:

- a Windows desktop application,
- multiple class libraries,
- NUnit unit and integration tests,
- UI automation tests,
- an installer project.

Builds run on self-hosted Windows agents.

UI automation tests require a separate Windows agent with an interactive desktop session.

The installer consumes the compiled application artifacts and produces a release package.

Deliverables

1. Azure DevOps YAML Pipeline

Create an Azure DevOps YAML pipeline that demonstrates how you would build, validate, package, and prepare the application for release.

The pipeline should include:

- appropriate pipeline triggers,
- dependency restore,
- .NET build,
- NUnit test execution,
- test result publishing,
- UI tests executed on the dedicated interactive Windows agent,
- installer creation,
- artifact publishing.

You may use placeholder names for agent pools, service connections, feeds, paths, and environments.

Nice-to-have topics you may include:

- versioning strategy,
- quality checks,
- security checks,
- code signing,
- release or deployment stages,
- reusable YAML templates,
- release approvals or controls,
- separation between CI, validation, and release pipelines.

2. Concept Document

Provide a short Markdown document with a maximum of two pages explaining your approach.

The document should cover:

- the overall CI/CD flow,
- key design decisions,
- branching, pull request, and release strategy,
- build, testing, packaging, and artifact handling,
- handling of secrets, permissions, and service connections,
- where software protection, licensing, installer packaging, and update infrastructure fit into the process,
- how logging, telemetry, or monitoring could improve reliability,
- assumptions made for the assignment,
- possible improvements for a real production setup.

What We Are Looking For

We are primarily interested in your reasoning and design decisions. The submission should demonstrate practical understanding of Azure DevOps pipelines, Git workflows, testing, artifact management, security and release preparation for a commercial .NET desktop application.

We value clarity and thoughtful trade-offs more than a fully complete implementation. The solution does not need to run against a real Azure DevOps organization. However, it should be syntactically plausible, internally consistent, and realistic enough to demonstrate your intended design.